Equivalence between axiomatized congruence reasoning and equality-like introduction rules

Dale Miller

doi:10.4230/LIPIcs.FSCD.2026.24; CC BY4.0

Editorial note. Official published CC BY4.0 source; selected mathematical text and every specialized author proof continuation faithfully transcribed to editable TeX.

Editorial note. The two directions of the proof-system equivalence count once. All source conditions and necessary inference diagrams retained; no new inference rule or corrected author variable introduced.

Editorial note. Only independent inference-rule layout changed to fit the wrapper. Original PDF and original-layout transcription preserved. No mathematical certification or local compilation.

Editorial scope. Complete author material collection for one structural theorem; author retains responsibility for the mathematics.

Allowed standard prerequisites (editorial).

- Gentzen LJ cut and instantiation elimination. Exact author LJ rules retained; standard admissibility facts invoked in original source.
- Synthetic inference rules induced by equivalence/compatibility axioms. Exact source synthetic rules retained with original author references[26,33].

The Basics: Types, terms, formulas, sequents, theories

Following Church [8], we base the syntax of terms and formulas on the simply typed λ-calculus. We reserve the primitive type o, and assume that there is a fixed set of primitive types (also called sorts) $\mathbb{S}$ that do not contain o. When $\sigma \in \mathbb{S}$, we shall also use σ as a predicate of type $\sigma \to o$. Our first-order, multi-sorted intuitionistic logic is based on the following propositional connectives

$$\mathsf{t} : o, \quad \wedge : o \to o \to o, \quad \mathsf{f} : o, \quad \vee : o \to o \to o, \quad \supset : o \to o \to o,$$

and the following connectives involve term structure (here, $\sigma \in \mathbb{S}$)

$$=_\sigma : \sigma \to \sigma \to o, \quad \forall_\sigma : (\sigma \to o) \to o, \quad \exists_\sigma : (\sigma \to o) \to o.$$

(We shall occasionally omit writing a typing subscript on an equality or quantifier when it can be inferred from context.) We work only in a first-order subset of Church's framework by restricting the types assigned to non-logical constants as follows.

- $P : \sigma_1 \to \cdots \to \sigma_n \to o$ is an n-ary predicate symbol and $\{\sigma_1, \ldots, \sigma_n\} \subseteq \mathbb{S}$ $(n \geq 0)$.
- $f : \sigma_1 \to \cdots \to \sigma_n \to \sigma_0$ is an n-ary constructor of type σ_0 and $\{\sigma_0, \ldots, \sigma_n\} \subseteq \mathbb{S}$ $(n \geq 0)$.
- $\sigma : \sigma \to o$ is a unary predicate symbol reflecting the sort σ also as a predicate.

A collection of such declarations is called a first-order signature. We usually write Σ_0 to denote the signature of non-logical constants involved in any particular logical specification. A Σ_0-formula is any closed term of type o, all of whose non-logical constants come from Σ_0. Note that if Σ_0 contains declarations of only predicate symbols, then a Σ_0-formula contains no occurrences of constructors, even of primitive arity. Sequents are two-sided and denoted by $\Sigma :: \Gamma \vdash B$, where the left-hand side Γ is a set of formulas, and the right-hand side B is a single formula. The signature Σ declares the types from $\mathbb{S}$ for all free variables occurring in $\{B\} \cup \Gamma$. Following Gentzen's terminology, Σ is best understood as declaring the eigenvariables of the sequent—variables that function as generic or "freshly picked" parameters. Formally, if Σ_0 is the fixed signature of non-logical constants (constructors and predicates), then all formulas in the sequent are $(\Sigma_0 \cup \Sigma)$-formulas. However, since Σ_0 is typically held constant, we usually omit it and simply refer to these as Σ-formulas, implying that their free variables are strictly contained within Σ. The usual inference rules for LJ are listed in Figure 1. Both the *instan* and *cut* rules are eliminable, in the sense that if a sequent is provable in LJ then it is provable in LJ without these rules. Theories are finite sets of formulas. We shall often informally say that some formula, say B, holds, while what we generally mean is that B is provable from some theory that might be left implicit. If C is a formula whose free variables are in the list $\bar{x}$, we will write this formula as $B\bar{x}$ where B is the (closed) abstraction $\lambda\bar{x}.C$. Here, λ plays only a notational role and does not need to be seen as part of our first-order logic.

Figure 1. The inference rules for the LJ proof system (as printed).

$$\frac{}{\Sigma :: \Gamma \vdash \mathsf{t}}\ \mathsf{t}R$$

$$\frac{\Sigma :: \Gamma \vdash B \quad \Sigma :: \Gamma \vdash C}{\Sigma :: \Gamma \vdash B \wedge C}\ \wedge R$$

$$\frac{\Sigma :: \Gamma \vdash B_i}{\Sigma :: \Gamma \vdash B_1 \vee B_2}\ \vee R,\ i \in \{1, 2\}.$$

$$\frac{\Sigma :: B, \Gamma \vdash C}{\Sigma :: \Gamma \vdash B \supset C}\ \supset R$$

$$\frac{y : \sigma, \Sigma :: \Gamma \vdash B[y/x]}{\Sigma :: \Gamma \vdash \forall_\sigma x.B}\ \forall R$$

$$\frac{\Sigma \vdash t : \sigma \quad \Sigma :: \Gamma \vdash B[t/x]}{\Sigma :: \Gamma \vdash \exists_\sigma x.B}\ \exists R$$

$$\frac{\Sigma :: B_i, \Gamma \vdash C}{\Sigma :: B_1 \wedge B_2, \Gamma \vdash C}\ \wedge L_i,\ i \in \{1,2\}.$$

$$\frac{\Sigma \vdash t : \sigma \quad \Sigma :: B[t/x], \Gamma \vdash C}{\Sigma :: \forall_\sigma x.B, \Gamma \vdash C}\ \forall L$$

$$\frac{\Sigma :: \Gamma_1 \vdash B \quad \Sigma :: C, \Gamma_2 \vdash C}{\Sigma :: B \supset C, \Gamma_1, \Gamma_2 \vdash C}\ \supset L$$

$$\frac{\Sigma :: B, \Gamma \vdash C \quad \Sigma :: C, \Gamma \vdash C}{\Sigma :: B \vee C, \Gamma \vdash C}\ \vee L$$

$$\frac{}{\Sigma :: \mathsf{f}, \Gamma \vdash C}\ \mathsf{f}L$$

$$\frac{}{\Sigma :: B, \Gamma \vdash B}\ \mathit{init}$$

$$\frac{\Sigma :: \Gamma \vdash B \quad \Sigma :: \Gamma, B \vdash E}{\Sigma :: \Gamma \vdash E}\ \mathit{cut}$$

$$\frac{\Sigma \vdash t : \sigma \quad \Sigma, x : \sigma :: \Gamma \vdash B}{\Sigma :: \Gamma[t/x] \vdash B[t/x]}\ \mathit{instan}$$

Figure 2. The rules to add to LJ to obtain the $LJ^=$ proof system.

$$\frac{}{\Sigma :: \Gamma \vdash t = t}\ =_R$$

$$\frac{(t \text{ and } s \text{ not unifiable})}{\Sigma :: s = t, \Gamma \vdash C}\ =_L$$

$$\frac{(\theta = \mathrm{mgu}(t,s)) \quad \Sigma\theta :: \Gamma\theta \vdash C\theta}{\Sigma :: s = t, \Gamma \vdash C}\ =_L$$

An invariant of the sequents considered in this paper is that, for any sequent $\Sigma :: \Gamma \vdash B$, a variable declaration $x : \sigma$ appears in Σ if and only if the atomic formula σx appears in Γ. This property—the typing redundancy invariant—is preserved provided that the quantifiers $\forall_\sigma x.B$ and $\exists_\sigma x.B$ are treated as abbreviations for $\forall x_\sigma.\sigma x \supset B$ and $\exists x_\sigma.\sigma x \wedge B$, respectively.

Inference rules for equality

The sequent calculus proof rules for the introduction of equality are presented in Figure 2. In the two versions of the $=_L$ rule, unification is done with respect to the eigenvariables. The signature $\Sigma\theta$ is the result of removing from Σ all variables in the domain of θ and adding in all variables free in a term in the domain. When these rules are added to the LJ proof system, we get the proof system $LJ^=$. It is important to understand that equality is a logical connective and that these inference rules fix its meaning. The addition of axioms does not affect this notion of equality. Since we shall reserve the term "atomic formula" for formulas that are not top-level logical connectives, a formula such as $t = s$ is not an atomic formula. A formula is equality-free if it contains no occurrences of equality.

Equivalences and compatibilities

For every primitive type $\sigma \in \mathbb{S}$, we assume that we have a binary predicate eq_σ: that is, a non-logical constant of type $\sigma \to \sigma \to o$. We abbreviate by $\mathbb{E}$ the set $\{\mathsf{eq}_\sigma\}_{\sigma\in\mathbb{S}}$ of such binary predicates and by $\mathcal{E}$ the set of formulas that axiomatize the properties of reflexivity, symmetry, and transitivity for all the predicates in $\mathbb{E}$. Given this axiomatization, these binary predicates denote equivalence relations. Let P be an abstraction with type $\sigma_1 \to \cdots \to \sigma_n \to o$ where $n \geq 0$. We say that P is compatible with respect to $\mathbb{E}$ (following terminology in, say, [7]), if the following formula holds.

$$\text{Compatibility:} \qquad \forall\bar{x}\forall\bar{y}.\Big(\bigwedge_{i=1}^{n} \mathsf{eq}\, x_i\, y_i\Big) \supset P\bar{x} \supset P\bar{y}.$$

The equivalence predicates eq_σ themselves are trivial examples of predicates that are compatible with $\mathbb{E}$. Let $\mathbb{C}$ be a set of predicates and define $\mathcal{C}$ to be the set of formulas that axiomatize the compatibility of the predicates in $\mathbb{C}$. The following theorem proves that any abstraction over formulas whose only non-logical constants are compatible with $\mathbb{E}$ is also similarly compatible. The theorems in this section consider only formulas that are equality-free: this is necessary since there might be a type $\sigma \in \mathbb{S}$ such that eq_σ is different from $=_\sigma$: this would cause a problem since we intend to lift eq_σ to be equality in a certain sense. It is, however, perfectly possible for eq_σ to be the same as $=_\sigma$. Equality plays a role starting in the next section when we define determinacy.

Theorem 1. Let B be an abstraction of type $\sigma_1 \to \cdots \to \sigma_n \to o$ $(n \geq 0)$ of an equality-free formula for which all non-logical symbols are in $\mathbb{E} \cup \mathbb{C}$. The following compatibility-like property is provable in $LJ^=$:

$$\mathcal{E}, \mathcal{C} \vdash \forall\bar{x}\bar{y}.\Big(\bigwedge_{i} \mathsf{eq}\, x_i\, y_i\Big) \supset B\bar{x} \supset B\bar{y}.$$

Proof. Since the displayed sequent above has no occurrences of equality, it is provable in $LJ^=$ if and only if the sequent

$$x_1, y_1, \ldots, x_n, y_n :: \mathcal{E}, \mathcal{C}, \mathsf{eq}\, x_1\, y_1, \ldots, \mathsf{eq}\, x_n\, y_n, B\bar{x} \vdash B\bar{y}$$

is provable in LJ. We proceed by induction on the structure of $B\bar{x}$ to show that this sequent is provable. Besides being equality-free, $B\bar{x}$ will necessarily contain no term constructors. Our proof is a simple modification of the proof that the general initial rule is admissible when only atomic initial rules are available.

Case. $B\bar{x}$ is an atom. There are three kinds of atomic formulas possible here.

1. If $B\bar{x}$ is of the form $\mathsf{eq}_\sigma x_i x_j$, then the proof is completed using symmetry and transitivity for eq_σ.
2. If $B\bar{x}$ is of the form σx_i then we know that y_i is of the same type as x_i and, given the typing redundancy of sequents, we know that $\sigma y_i \in \Gamma$.
3. The remaining case is when this atom is of the form $(p\, u_1 \cdots u_i)$ for $i \geq 0$, $p \in \mathbb{C}$, and $u_1, \ldots, u_i$ is a list of variables in $\bar{x}$. This case is proved directly by using the compatibility property for p that appears in $\mathcal{C}$.

Case. $B\bar{x}$ *is* $B_1\bar{x} \supset B_2\bar{x}$. The inductive hypotheses give proofs of two sequents

$$x_1, y_1, \ldots, x_n, y_n :: \mathcal{E}, \mathcal{C}, \mathsf{eq}\, x_1\, y_1, \ldots, \mathsf{eq}\, x_n\, y_n, B_1\bar{y} \vdash B_1\bar{x},$$
$$x_1, y_1, \ldots, x_n, y_n :: \mathcal{E}, \mathcal{C}, \mathsf{eq}\, x_1\, y_1, \ldots, \mathsf{eq}\, x_n\, y_n, B_2\bar{x} \vdash B_2\bar{y}.$$

The proof of the first of these also requires the symmetry property of the equivalence relations. From these two subproofs, one can immediately build a proof of

$$x_1, y_1, \ldots, x_n, y_n :: \mathcal{E}, \mathcal{C}, \mathsf{eq}\, x_1\, y_1, \ldots, \mathsf{eq}\, x_n\, y_n, B_1\bar{x} \supset B_2\bar{x} \vdash B_1\bar{x} \supset B_2\bar{x}.$$

The cases for the other binary propositional connectives ($\wedge$ and $\vee$) are completely similar. The cases for t and f are immediate.

Case. $B\bar{x}$ *is* $\forall u, B_0 u\bar{x}$ *where* B_0 *is an* $n+1$ *abstraction.* The inductive hypothesis yields a proof of

$$x_1, y_1, \ldots, x_n, y_n, u, v :: \mathcal{E}, \mathcal{C}, \mathsf{eq}\, x_1\, y_1, \ldots, \mathsf{eq}\, x_n\, y_n, \mathsf{eq}\, u\, v, B_0 u\bar{x} \vdash B_0 v\bar{y}.$$

By substituting u for v into this sequent, cutting with the proof of $\mathsf{eq}\, u\, u$, and introduction rules for $\forall$ on the left and right, we have a proof of

$$x_1, y_1, \ldots, x_n, y_n, u, v :: \mathcal{E}, \mathcal{C}, \mathsf{eq}\, x_1\, y_1, \ldots, \mathsf{eq}\, x_n\, y_n, \forall u. B_0 u\bar{x} \vdash \forall u. B_0 u\bar{y}.$$

The case for the existential quantifier is similar. □

It is useful at times to replace the formulas in $\mathcal{E}$ and $\mathcal{C}$ with synthetic rules. By employing the techniques found in [26,33], these formulas can also be turned into the following inference rules, encoding, respectively, reflexivity, symmetry, transitivity, and compatibility.

$$\frac{u : \sigma, \Sigma :: \mathsf{eq}_\sigma uu, \Gamma \vdash B}{u : \sigma, \Sigma :: \Gamma \vdash B}$$

$$\frac{\Sigma :: \mathsf{eq}_\sigma ts, \mathsf{eq}_\sigma st, \Gamma \vdash B}{\Sigma :: \mathsf{eq}_\sigma ts, \Gamma \vdash B}$$

$$\frac{\Sigma :: \mathsf{eq}_\sigma ts, \mathsf{eq}_\sigma sr, \mathsf{eq}_\sigma tr, \Gamma \vdash B}{\Sigma :: \mathsf{eq}_\sigma ts, \mathsf{eq}_\sigma sr, \Gamma \vdash B}$$

$$\frac{\Sigma :: \mathsf{eq}_{\sigma_1} t_1 s_1, \ldots, \mathsf{eq}_{\sigma_1} t_1 s_1, P\, t_1 \cdots t_n, P\, s_1 \cdots s_n, \Gamma \vdash B}{\Sigma :: \mathsf{eq}_{\sigma_1} t_1 s_1, \ldots, \mathsf{eq}_{\sigma_1} t_1 s_1, P\, t_1 \cdots t_n, \Gamma \vdash B}$$

As shown in [26], the addition of such synthetic rules to LJ results in a proof system that satisfies the cut elimination theorem.

Inference rules for equivalences

If we are working only with equality-free formulas over $\mathbb{E} \cup \mathbb{C}$, then we can move to a new proof system in which the (possibly numerous) axioms describing compatibility and equivalence are replaced by inference rules that make the predicates in $\mathbb{E}$ into logical connectives by giving them their own left and right

introduction rules. In particular, those new introduction rules are essentially the rules for equality.

$$\frac{}{\Sigma :: \Gamma \vdash \mathsf{eq}_\sigma tt}\ \mathsf{eq}_\sigma R \qquad \frac{u, \Sigma :: \Gamma[u/v] \vdash B[u/v]}{u, v, \Sigma :: \mathsf{eq}_\sigma uv, \Gamma \vdash B}\ \mathsf{eq}_\sigma L$$

Given that we will be working with formulas without occurrences of constructors, the rules for introducing equivalences on the left and right are simpler than for general equality. In particular, the left introduction rule lacks the "failure-to-unify" case. Let $LJ_{\mathbb{E}}$ be the proof system that results from adding the above introduction rules (for every $\sigma \in \mathbb{S}$) to LJ, and let $LJ^{=}_{\mathbb{E}}$ be the proof system that results from adding them to $LJ^{=}$.

Theorem 2. Let B be an $\mathbb{E} \cup \mathbb{C}$-formula without equalities. If $\cdot :: \mathcal{E}, \mathcal{C} \vdash B$ has an $LJ^{=}$ proof then $\cdot :: \cdot \vdash B$ has an $LJ^{=}_{\mathbb{E}}$ proof.

Proof. Assume $\cdot :: \mathcal{E}, \mathcal{C} \vdash B$ has an $LJ^{=}$ proof. Since B contains no equalities, then $\cdot :: \mathcal{E}, \mathcal{C} \vdash B$ has an LJ proof. For the purpose of this proof, we extend the LJ proof system to the LJ^{+} proof system by adding the synthetic rules for all the formulas in $\mathcal{E} \cup \mathcal{C}$: as a result, the assumptions in $\mathcal{E} \cup \mathcal{C}$ can be removed from the sequent. In that extended proof system, every sequent is of the form

$$u_1 : \sigma_1, \ldots, u_n : \sigma_n :: S, R, \Gamma \vdash C, \tag{1}$$

where S is $\{\sigma_1 u_1, \ldots, \sigma_n u_n\}$, and R is the set of atomic formulas of the form $\mathsf{eq}_\sigma u_i u_j$ (and we assume that Γ does not contain such atomic formulas). For $\sigma \in \mathbb{S}$, let $\equiv_\sigma$ be the least equivalence relation on the variables in $\{u_1, \ldots, u_n\}$ that contains the relation $\{\langle u, v\rangle \mid \mathsf{eq}_\sigma uv \in R\}$. We fix some linear order on variables (say, alphabetic) and define the function $\widehat{\equiv}_\sigma$ from variables to variables (both of type σ) such that $\widehat{\equiv}_\sigma(u) = v$ if v is the representative of the $\equiv_\sigma$-equivalence class containing u. Here, the representative is the member of the equivalence class that is least in the order relation. The function $\widehat{\equiv}(C)$ is defined to map the formula C to the formula that results in substituting free variables of C (of type, say, σ) with their image under $\widehat{\equiv}_\sigma$. Next, we apply $\widehat{\equiv}$ to the formulas in the sequent (1) to get the new sequent

$$v_1, \ldots, v_m :: \widehat{\equiv}_\sigma(S), \widehat{\equiv}_\sigma(R), \widehat{\equiv}_\sigma(\Gamma) \vdash \widehat{\equiv}_\sigma(C). \tag{2}$$

Here, the v_i's are representatives of equivalence classes and the atoms in $\widehat{\equiv}_\sigma(R)$ are all of the form $\mathsf{eq}_\sigma v_i v_i$. We shall say that the sequent (2) is associated with the sequent (1). A simple induction on the structure of LJ^{+} proofs shows that if (1) has an LJ^{+} proof, then its associated sequent has an $LJ_{\mathbb{E}}$ proof. As a result of this induction, we know that since the sequent $\cdot :: \cdot \vdash B$ has an $LJ^{=}$ proof, then the associated sequent $\cdot :: \cdot \vdash B$ has an $LJ^{=}_{\mathbb{E}}$ proof. □

Theorem 3. Let B be an $\mathbb{E} \cup \mathbb{C}$-formula without equalities. If $\cdot :: \cdot \vdash B$ has an $LJ^{=}_{\mathbb{E}}$ proof then $\cdot :: \mathcal{E}, \mathcal{C} \vdash B$ has an $LJ^{=}$ proof.

Proof. We show how to transform a $LJ^{=}_{\mathbb{E}}$ proof of $\Sigma :: \Gamma \vdash B$ into an $LJ^{=}$ proof $\Sigma :: \mathcal{E}, \mathcal{C}, \Gamma \vdash B$. All rules of $LJ^{=}_{\mathbb{E}}$ are rules of $LJ^{=}$ except for the $\mathsf{eq}_\sigma L$ and $\mathsf{eq}_\sigma R$

rules. The $\mathsf{eq}_\sigma R$ rule is easily accounted for using the reflective rules in $\mathcal{E}$. To account for the $\mathsf{eq}_\sigma L$ rule, we first prove that the sequents

$$u, v, \Sigma :: \mathcal{E}, \mathcal{C}, \mathsf{eq}_\sigma uv, B \vdash B[u/v] \tag{3}$$

$$u, v, \Sigma :: \mathcal{E}, \mathcal{C}, \mathsf{eq}_\sigma uv, B[u/v] \vdash B \tag{4}$$

have $LJ^=$ proofs. We do this by induction on the structure of B. The case where B is an atomic formula follows immediately from the axioms in $\mathcal{E}, \mathcal{C}$. All the other cases follow directly from the inductive hypothesis. Now, assume that the last inference rule is the $\mathsf{eq}_\sigma L$ rule. We apply the inductive assumption to conclude that the sequent

$$u, \Sigma :: \mathcal{E}, \mathcal{C}, \Gamma[u/v] \vdash B[u/v]$$

has an $LJ^=$ proof. We can now apply the cut rule with (3) for every member of $\Gamma[u/v]$ and with (4) with $B[u/v]$. Cut elimination for $LJ^=$ proofs then yields the desired proof of the sequent $u, v, \Sigma :: \mathcal{E}, \mathcal{C}, \mathsf{eq}_\sigma uv, \Gamma \vdash B$. □

References

[1] Peter B. Andrews. An Introduction to Mathematical Logic and Type Theory: To Truth Through Proof, volume 27 of Applied Logic Series. Kluwer Academic Publishers, second edition, 2002. doi:10.1007/978-94-015-9934-4.

[2] Ali Assaf, Guillaume Burel, Raphaël Cauderlier, David Delahaye, Gilles Dowek, Catherine Dubois, Frédéric Gilbert, Pierre Halmagrand, Olivier Hermant, and Ronan Saillard. Dedukti: a Logical Framework based on the λΠ-calculus Modulo Theory, September 2016. Working Paper or Preprint. URL: https://hal.science/hal-01368161.

[3] Leo Bachmair and Harald Ganzinger. Rewrite-based equational theorem proving with selection and simplification. Journal of Logic and Computation, 4(3):217–247, 1994. doi:10.1093/ logcom/4.3.217.

[4] David Baelde, Kaustuv Chaudhuri, Andrew Gacek, Dale Miller, Gopalan Nadathur, Alwen Tiu, and Yuting Wang. Abella: A system for reasoning about relational specifications. J. of Formalized Reasoning, 7(2):1–89, 2014. doi:10.6092/issn.1972-5787/4650.

[5] David Baelde, Andrew Gacek, Dale Miller, Gopalan Nadathur, and Alwen Tiu. The Bedwyr system for model checking over syntactic expressions. In F. Pfenning, editor, 21th Conf. on Automated Deduction (CADE), number 4603 in LNAI, pages 391–397, New York, 2007. Springer. doi:10.1007/978-3-540-73595-3_28.

[6] Gilles Barthe, Venanzio Capretta, and Olivier Pons. Setoids in type theory. Journal of Functional Programming, 13(2):261–293, 2003. doi:10.1017/S0956796802004501.

[7] Stanley Burris and H. P. Sankappanavar. A Course in Universal Algebra, volume 78 of Graduate Texts in Mathematics. Springer-Verlag, 1981. Millennium Edition available online. doi:10.1007/978-1-4613-8130-3.

[8] Alonzo Church. A formulation of the Simple Theory of Types. J. of Symbolic Logic, 5:56–68, 1940. doi:10.2307/2266170.

[9] Claudio Sacerdoti Coen. A semi-reflexive tactic for (sub-)equational reasoning. In Jean- Christophe Filliâtre, Christine Paulin-Mohring, and Benjamin Werner, editors, Types for Proofs and Programs (TYPES 2004), volume 3839 of Lecture Notes in Computer Science, pages 98–114. Springer, 2006. doi:10.1007/11617990_7.

[10] Robert L. Constable et al. Implementing Mathematics with the Nuprl Proof Development System. Prentice-Hall, 1986.

[11] Gilles Dowek, Thérèse Hardin, and Claude Kirchner. Theorem proving modulo. J. of Automated Reasoning, 31(1):31–72, 2003.

[12] Cormac Flanagan, Amr Sabry, Bruce F. Duba, and Matthias Felleisen. The essence of compiling with continuations. ACM SIGPLAN Notices, 28(6):237–247, 1993. doi:10.1145/ 155090.155113.

[13] Peter J. Freyd. Properties invariant within equivalence types of categories. In Alex Heller and Myles Tierney, editors, Algebra, Topology, and Category Theory: A Collection of Papers in Honor of Samuel Eilenberg, pages 55–61. Academic Press, New York, 1976. doi:10.1016/ B978-0-12-339050-9.50010-0.

[14] Peter J. Freyd and Andre Scedrov. Categories, Allegories, volume 39 of North-Holland Mathematical Library. Elsevier Science Publishers B.V., Amsterdam, Netherlands, 1990. doi:10.1016/C2009-0-09144-1. 24:16 Treating Congruences as Equalities Within Proofs

[15] Andrew Gacek, Dale Miller, and Gopalan Nadathur. Nominal abstraction. Information and Computation, 209(1):48–73, 2011. doi:10.1016/j.ic.2010.09.004.

[16] Gerhard Gentzen. The consistency of elementary number theory. In M. E. Szabo, editor, The Collected Papers of Gerhard Gentzen, pages 132–213. North-Holland, Amsterdam, 1969. Translated from the 1936 German original. doi:10.1016/S0049-237X(08)70823-1.

[17] Ulysse Gérard and Dale Miller. Separating functional computation from relations. In Valentin Goranko and Mads Dam, editors, 26th EACSL Annual Conference on Computer Science Logic (CSL 2017), volume 82 of LIPIcs, pages 23:1–23:17, 2017. doi:10.4230/LIPIcs.CSL.2017.23.

[18] Jean-Yves Girard. A fixpoint theorem in linear logic. An email posting to linear@cs.stanford.edu archived at https://www.seas.upenn.edu/ sweirich/types/archive/1992/msg00030.html, February 1992.

[19] Joseph A. Goguen and José Meseguer. Equality, types, modules, and (why not?) generics for logic programming. The Journal of Logic Programming, 1(2):179–210, 1984. doi:10.1016/ 0743-1066(84)90011-X.

[20] Quentin Heath and Dale Miller. A proof theory for model checking. J. of Automated Reasoning, 63(4):857–885, 2019. doi:10.1007/s10817-018-9475-3.

[21] David Hilbert. The foundations of mathematics. In Jean van Heijenoort, editor, From Frege to Gödel: A Source Book in Mathematical Logic, 1879–1931, pages 464–479. Harvard University Press, Cambridge, Massachusetts, 1967. doi:10.4159/harvard.9780674324497.c22.

[22] Martin Hofmann and Thomas Streicher. The groupoid interpretation of type theory. In Twenty-Five Years of Constructive Type Theory, volume 36 of Oxford Logic Guides, pages 83–111. Oxford University Press, 1998. doi:10.1093/oso/9780198501275.003.0008.

[23] Oiva Ketonen. Investigations into the Predicate Calculus. College Publications, 2022. Ed. by S. Negri and J. von Plato.

[24] Laura Kovács and Andrei Voronkov. First-order theorem proving and Vampire. In Maria Paola Bonacina, editor, Automated Deduction - CADE-24 - 24th International Conference on Auto- mated Deduction, Lake Placid, NY, USA, June 9-14, 2013. Proceedings, volume 7898 of Lecture Notes in Computer Science, pages 1–35. Springer, 2013. doi:10.1007/978-3-642-38574-2_1.

[25] Michael Makkai. First order logic with dependent sorts, with applications to category theory. Manuscript, McGill University, 1995. URL: https://www.math.mcgill.ca/makkai/folds/ foldsinpdf/FOLDS.pdf.

[26] Sonia Marin, Dale Miller, Elaine Pimentel, and Marco Volpe. From axioms to synthetic inference rules via focusing. Annals of Pure and Applied Logic, 173(5):1–32, 2022. doi: 10.1016/j.apal.2022.103091.

[27] Per Martin-Löf. Constructive mathematics and computer programming. In Sixth International Congress for Logic, Methodology, and Philosophy of Science, pages 153–175, Amsterdam, 1982. North-Holland.

[28] Jia Meng and Lawrence C. Paulson. Translating higher-order clauses to first-order clauses. Journal of Automated Reasoning, 40(1):35–60, 2008. doi:10.1007/s10817-007-9085-y.

[29] Dale Miller and Gopalan Nadathur. Programming with Higher-Order Logic. Cambridge University Press, June 2012. doi:10.1017/CBO9781139021326.

[30] Dale Miller and Alwen Tiu. A proof theory for generic judgments. ACM Trans. on Computa- tional Logic, 6(4):749–783, October 2005. doi:10.1145/1094622.1094628.

[31] John C. Mitchell and Gordon D. Plotkin. Abstract types have existential type. ACM Transactions on Programming Languages and Systems, 10(3):470–502, 1988. doi:10.1145/ 44501.45065.

[32] Sara Negri and Jan von Plato. Cut elimination in the presence of axioms. Bulletin of Symbolic Logic, 4(4):418–435, 1998. doi:10.2307/420956.

[33] Sara Negri and Jan von Plato. Structural Proof Theory. Cambridge University Press, 2001.

[34] Sara Negri and Jan von Plato. Proof Analysis: A Contribution to Hilbert's Last Problem. Cambridge University Press, Cambridge, 2011. doi:10.1017/CBO9780511921629.

[35] Peter Schroeder-Heister. Rules of definitional reflection. In M. Vardi, editor, 8th Symp. on Logic in Computer Science, pages 222–232. IEEE Computer Society Press, IEEE, June 1993. doi:10.1109/LICS.1993.287585.

[36] Stephan Schulz, Simon Cruanes, and Petar Vukmirović. Faster, higher, stronger: E 2.3. 6th Vampire Workshop, 71:1–10, 2019. doi:10.29007/sc9t.

[37] Matthieu Sozeau. A new look at generalized rewriting in type theory. Journal of Formalized Reasoning, 2(1):41–62, 2009. doi:10.6092/issn.1972-5787/1574.